MASTER OF SCIENCE
IN ENGINEERING

Recurrent Neural Networks

Part 3: Seq2Seq - Attention mechanism

Jean Hennebert
Martin Melchior

Overview

1. Recaps

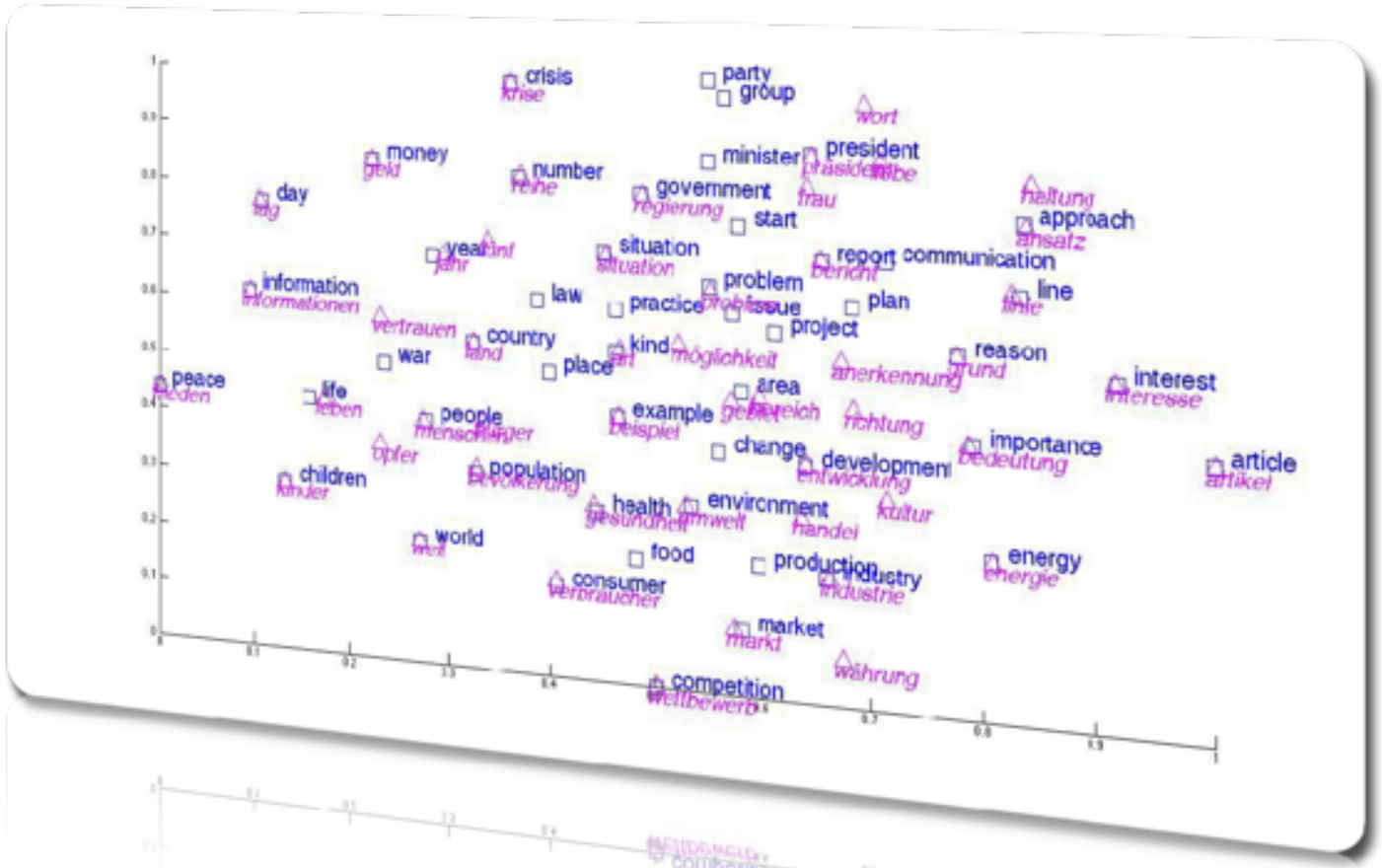
2. Seq2Seq models

3. Seq2Seq with attention

1. Recaps

Generative RNNs

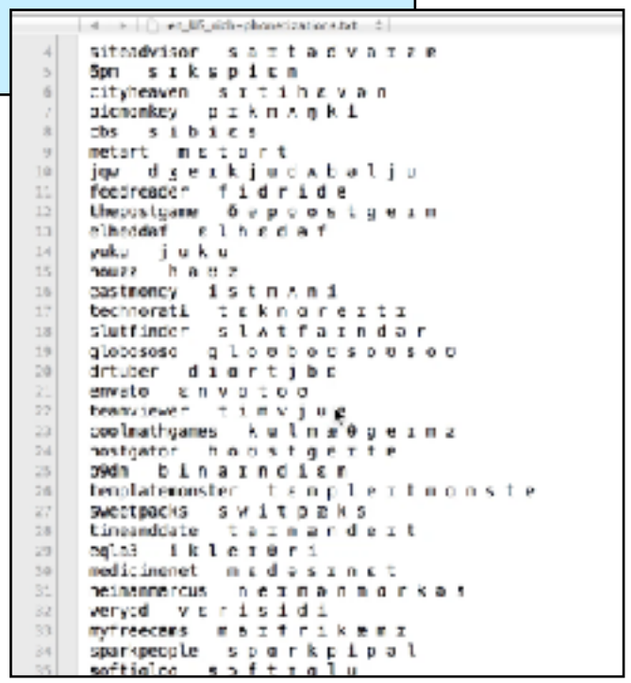
Word embeddings



Principle of data generation

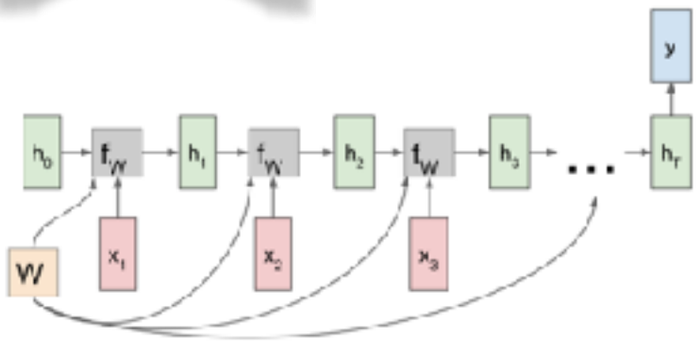
A **generative system** is a system able to generate new consistent data from a **seed**. By consistent, we mean respecting temporal or spatial “structures” that have been learned from the input space.

- Can be used to generate *sequential* data with RNNs
- 2 strategies can be used



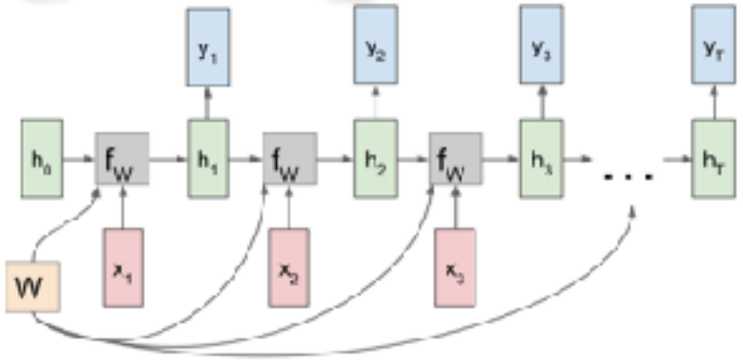
E.g. capture the regular patterns of phoneme sequences

Many to one



- The RNN is trained to predict the next token of the sequence.
- At testing time, a seed sequence is used and an output is sampled from the class probabilities.

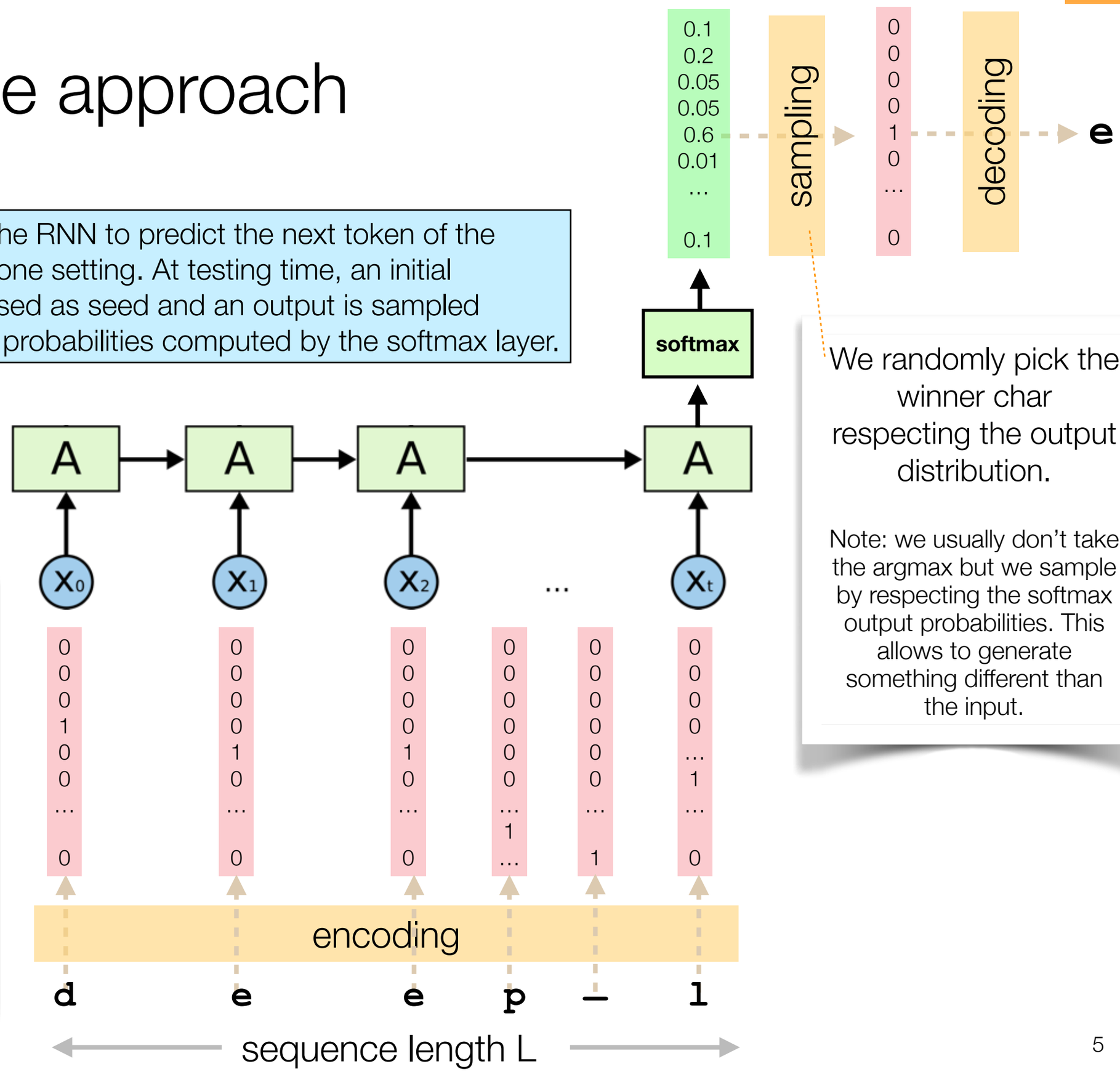
Many to many



- The output layer is “time distributed” to the whole sequence
- In Keras/TF: needs an extra layer “TimeDistributed”, also to handle the back propagation which is also distributed “through time”

Many to one approach

The principle is to train the RNN to predict the next token of the sequence in a many-to-one setting. At testing time, an initial sequence of tokens is used as seed and an output is sampled respecting the posterior probabilities computed by the softmax layer.



We randomly pick the winner char respecting the output distribution.

Note: we usually don't take the argmax but we sample by respecting the softmax output probabilities. This allows to generate something different than the input.

One-hot vector with a '1' at the position according to the position of the letter in the alphabet size D

```
{'a', 'b', 'c',…,
 'A', 'B', 'C',…,
 '-', "'",
'_','END'}
```

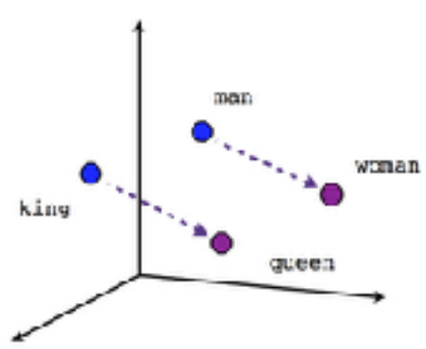
Word embedding - word2vec

Word embedding is a projection of a word into vectors of real numbers.

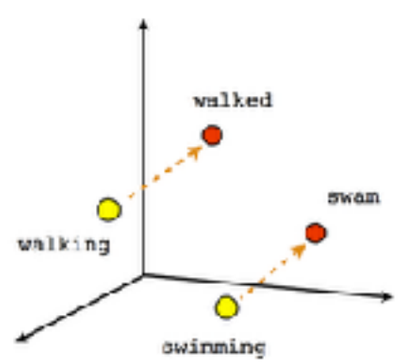
word2vec relates to models producing word embedding into space with **contextual similarity** or words, i.e. words that share common context are located in close proximity.

- The word2vec mapping is not only about clustering related words, it is also able to preserve some **"path meaning"**

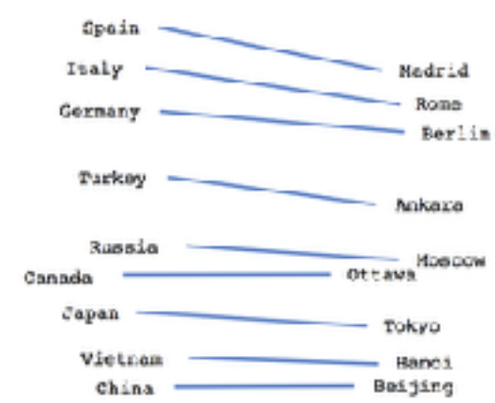
In vector terms:
$v_{woman} + (v_{king} - v_{man})$
is close to v_{queen}



Male-Female



Verb tense

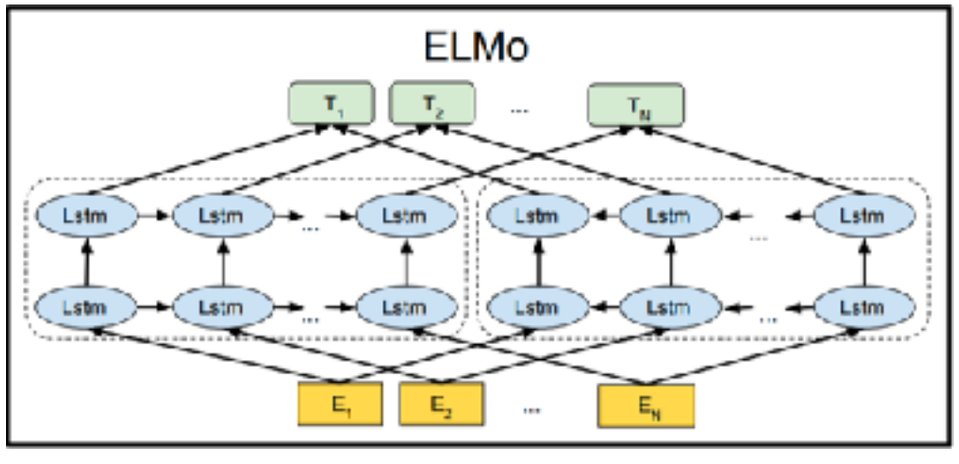


Country-Capital

Word embedding - FastText - ELMo

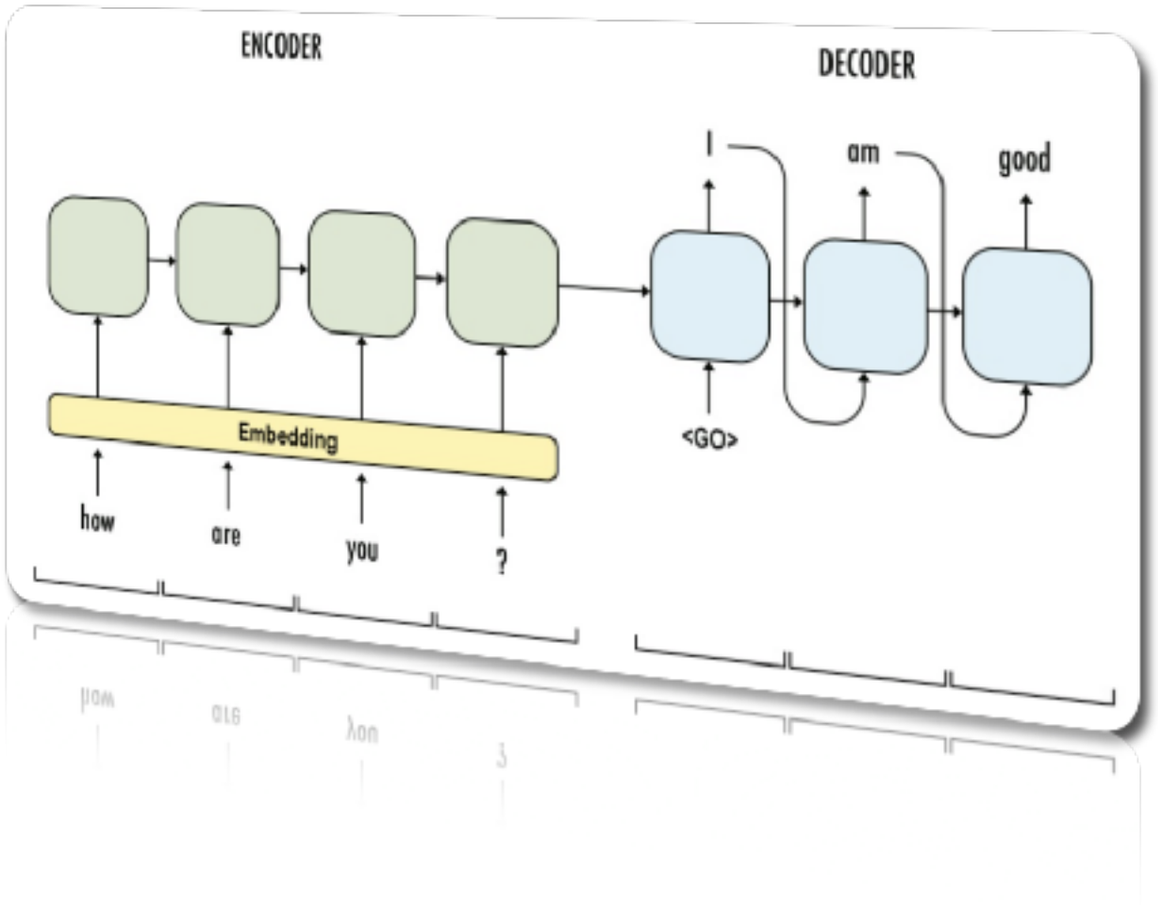
- FastText is an extension to Word2Vec
  - Instead of using individual words, FastText breaks words into several *n*-grams (i.e. sub-words)
  - The word embedding vector for a word will be the sum of all of its *n*-grams
  - Good to treat out-of-vocabulary words, good for languages using compound words
- ELMo = embeddings for *language models*
  - Intuition: meaning changes with context
  - The embedding is done after looking at the whole sentence

The term "word embedding" leaves the place to "**language models**".



Looking at the whole sentence is maybe overkilling… can we do better?
Attention mechanism

2. Seq2Seq Models

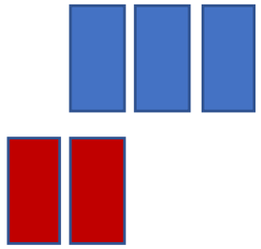


Sequence to sequence model - Seq2Seq

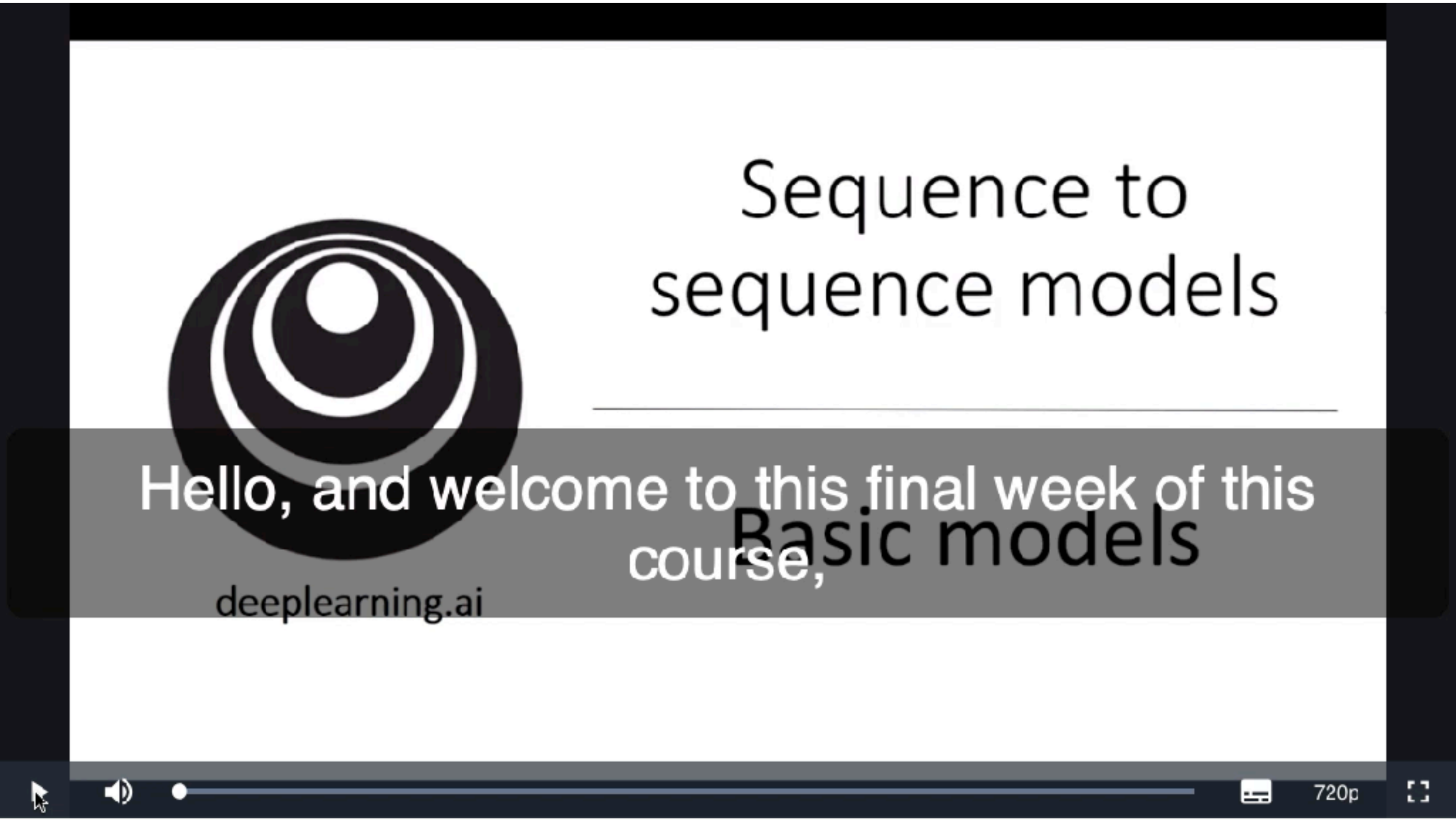
A **sequence to sequence** model is learning to map sequences of different length in situations where the entire input sequence is required in order to start predicting the target sequence.

- Typical applications are machine translation, question-answering bots, grapheme to phoneme mappers, etc
- Initial architectures were based on RNNs
  - 2 RNNs are actually used : encoder and decoder parts

many-to-many



$$T_x \geq 1, T_y \geq 1$$
$$T_x \neq T_y$$



https://youtu.be/_i3aqgKVNQI

Encoder - Decoder principle

- An **encoder** processes the input sequence and compresses the information into a context vector h_t of a fixed length
  - A “numerical summary” of the input sequence
    - Said “sentence embedding” or “thought” vector when inputs are words.
  - This representation is expected to be a good summary of the meaning of the whole source sequence.
- A **decoder** is initialized with the context vector to emit the transformed output.
  - In its original form, Seq2Seq only used the last state of the encoder network as the decoder initial state.
  - The generation is done in a similar way as for generative model (see previous class)

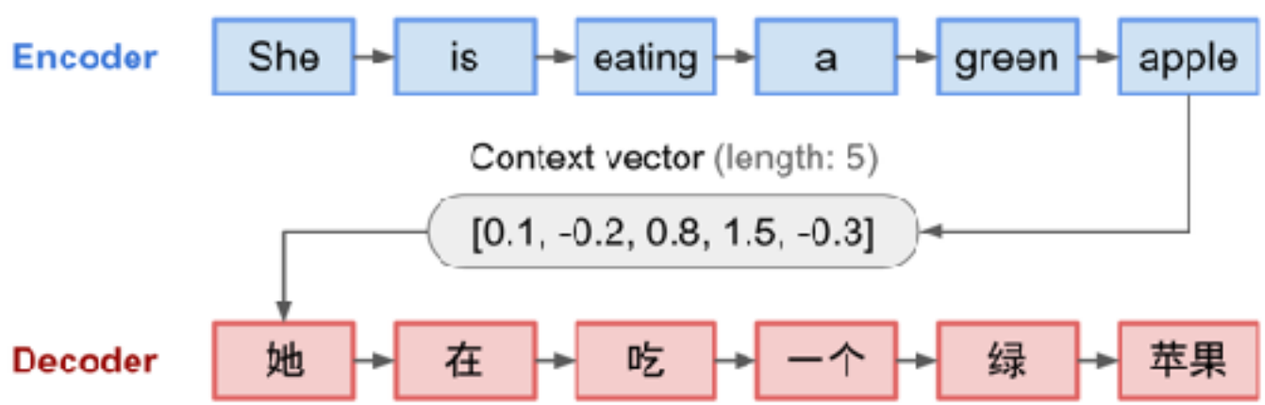
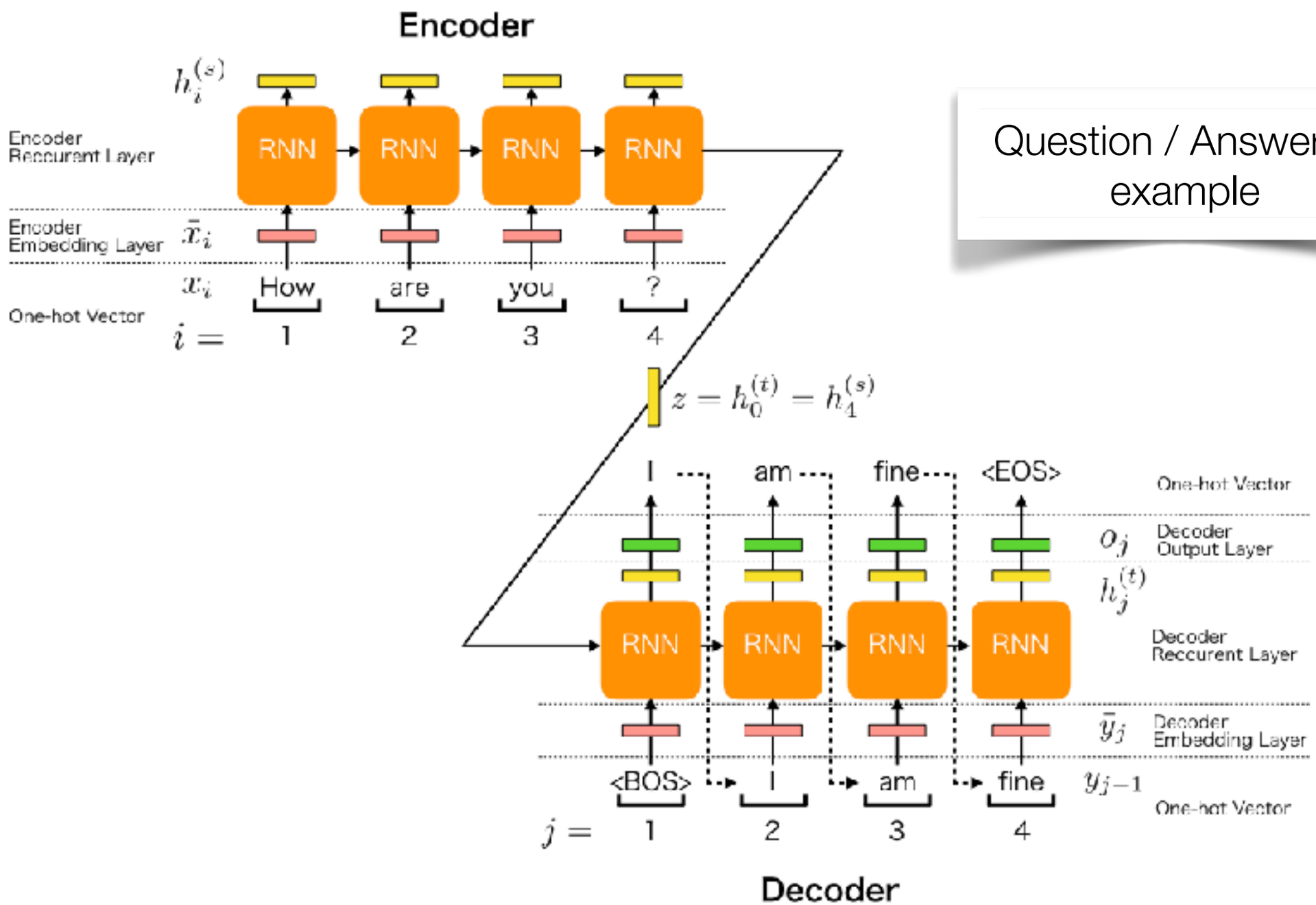


Image borrowed from: https://lilianweng.github.io/lil-log/2018/06/24/attention-attention.html

Typical encoder - decoder architecture



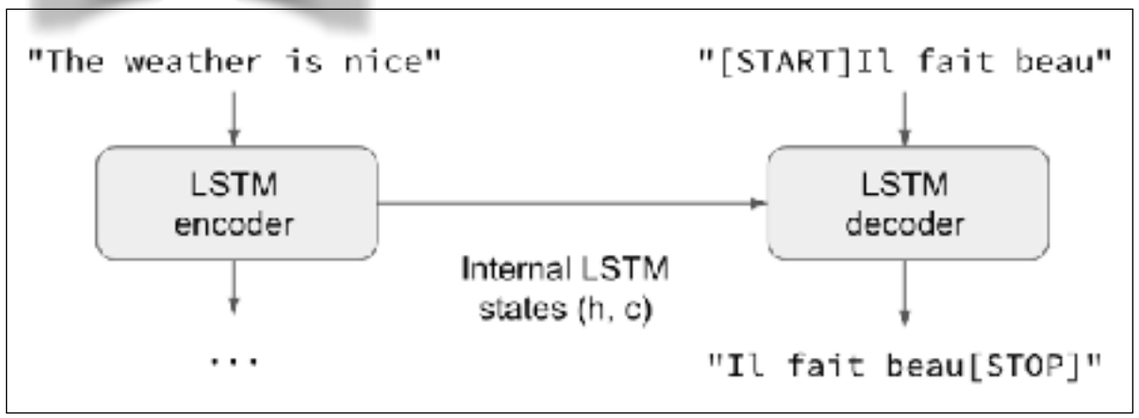
Question / Answering example

Image borrowed from: https://docs.chainer.org/en/stable/examples/seq2seq.html

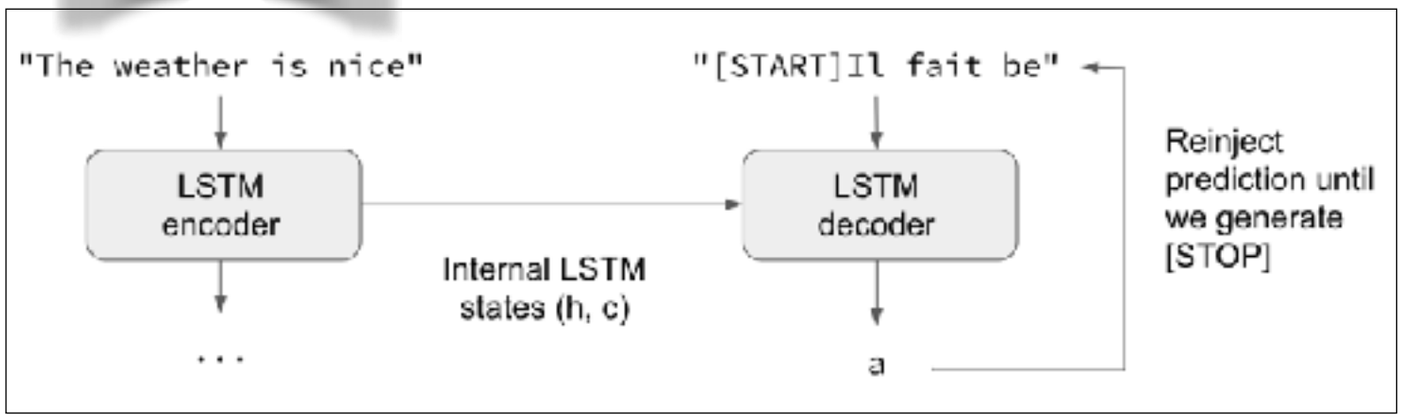
See practical work

Seq2Seq example for machine translation

Training time



Inference time

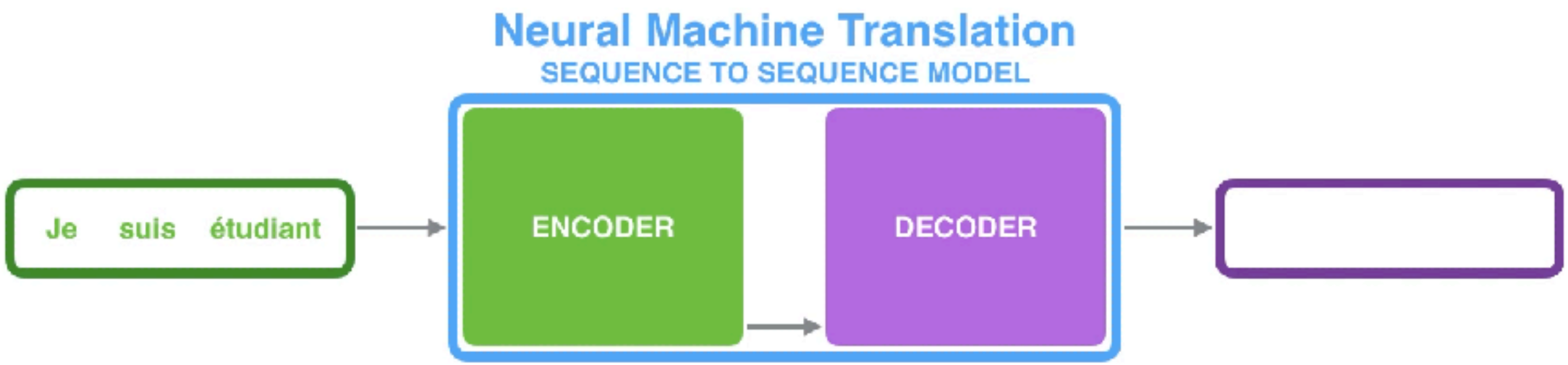


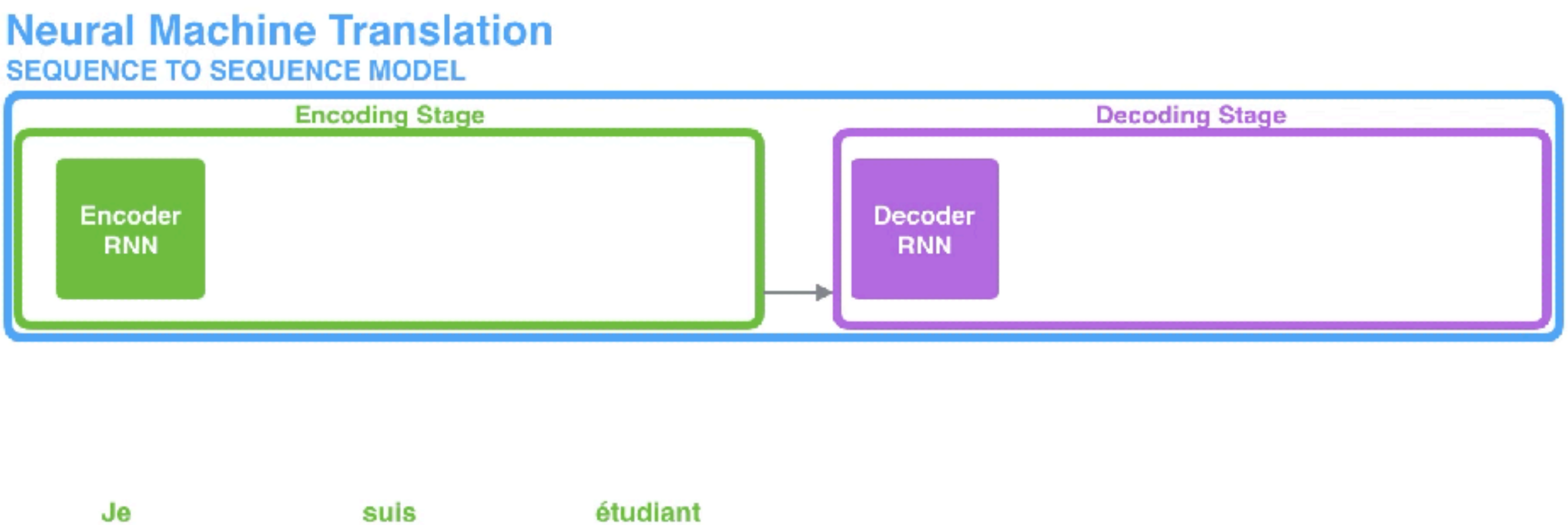

```
Input sentence: Be nice.
Decoded sentence: Soyez gentil !

Input sentence: Drop it!
Decoded sentence: Laissez tomber !

Input sentence: Get out!
Decoded sentence: Sortez !
```

- Basic character-level Seq2Seq model.
  - Applied to translating short English sentences into short French sentences, character-by-character.
  - Note that it is fairly unusual to do character-level machine translation, as word-level models are more common in this domain.
  - **Teacher forcing** is used for the training of the decoder: predict the next characters of the target sequence, given previous characters of the target sequence. In other words, it is trained to turn the target sequences into the same sequences but offset by one timestep in the future.
  - To be noted: special characters are used to mean *start* and *stop* of sentences, they are encoded as 1 char each.
- Explanations
  - https://blog.keras.io/a-ten-minute-introduction-to-sequence-to-sequence-learning-in-keras.html
  - https://keras.io/examples/nlp/lstm_seq2seq/
- Source code
  - https://github.com/keras-team/keras-io/blob/master/examples/nlp/lstm_seq2seq.py

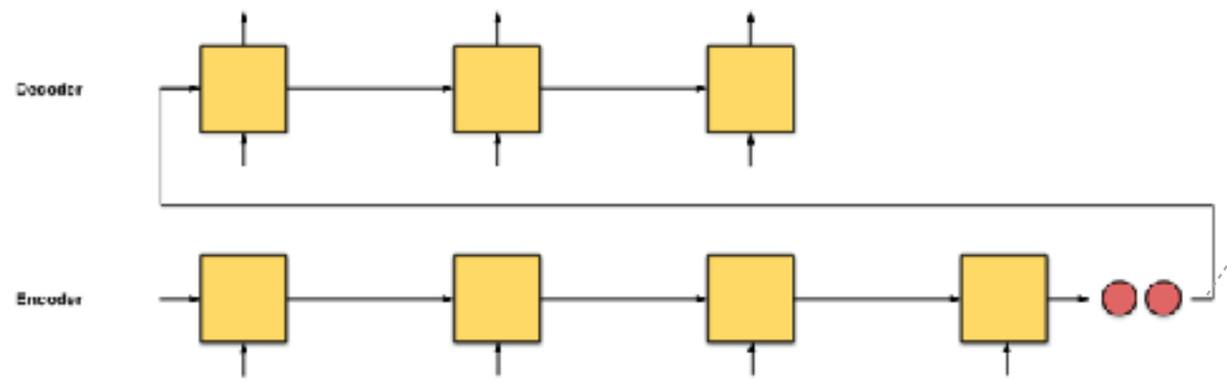




Video borrowed from https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

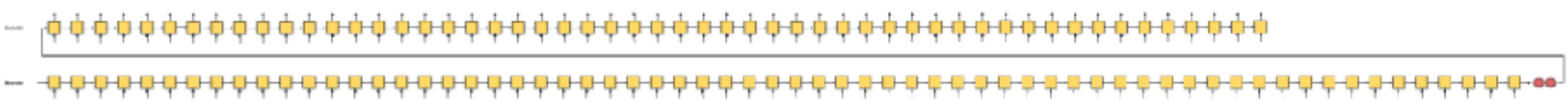
Problem of basic Seq2Seq model

- Mapping is ok for short sequences



Last encoder hidden state

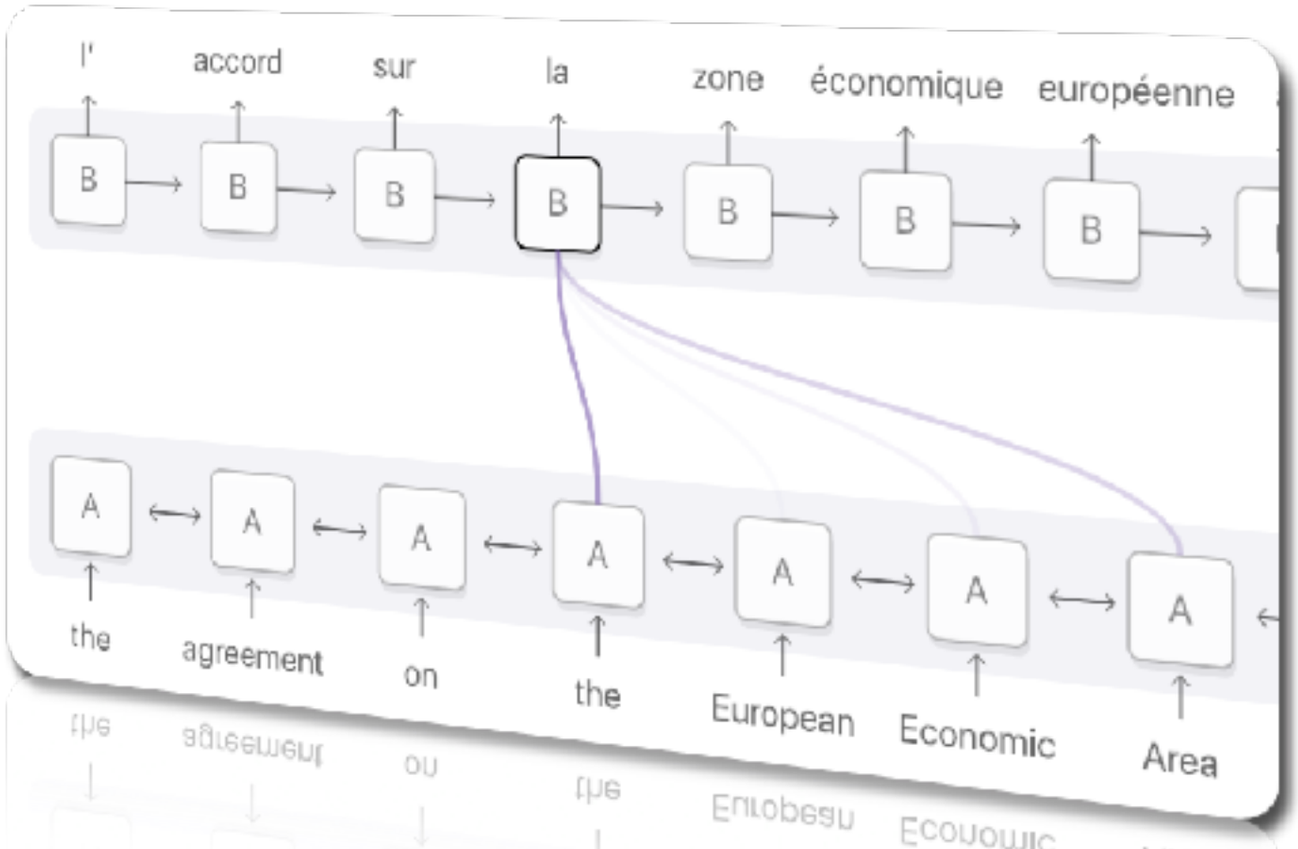
- Mapping is more difficult for long sequences
  - The context vector needs to “encode” a lot of information



- A solution is to plug-in **attention** mechanisms

Images borrowed from: https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

3. Seq2Seq with Attention



Attention

Attention is an interface between the encoder and decoder that provides the decoder with information from all (or parts of) the encoder hidden states.

- The encoder, without attention, receives only the hidden state h_t (in red below).

Two types of attention. **Global**: uses all the encoder hidden states. **Local**: uses only a subset of the encoder hidden states.

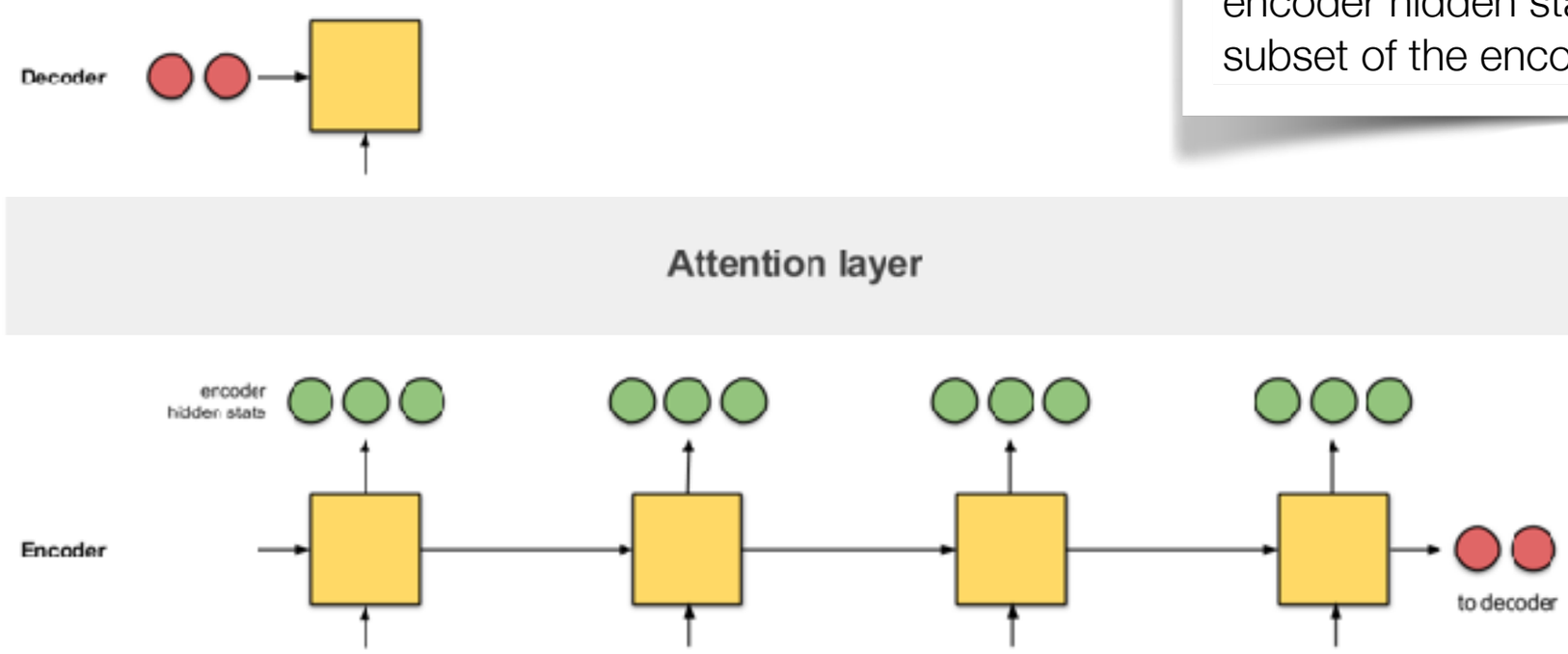


Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - intuition

- We want to provide the decoder with a notion of **alignment**
  - Which segments of original sequence match with their corresponding segments of the translation.
  - Where should the decoder “brings its attention” to map better the current output

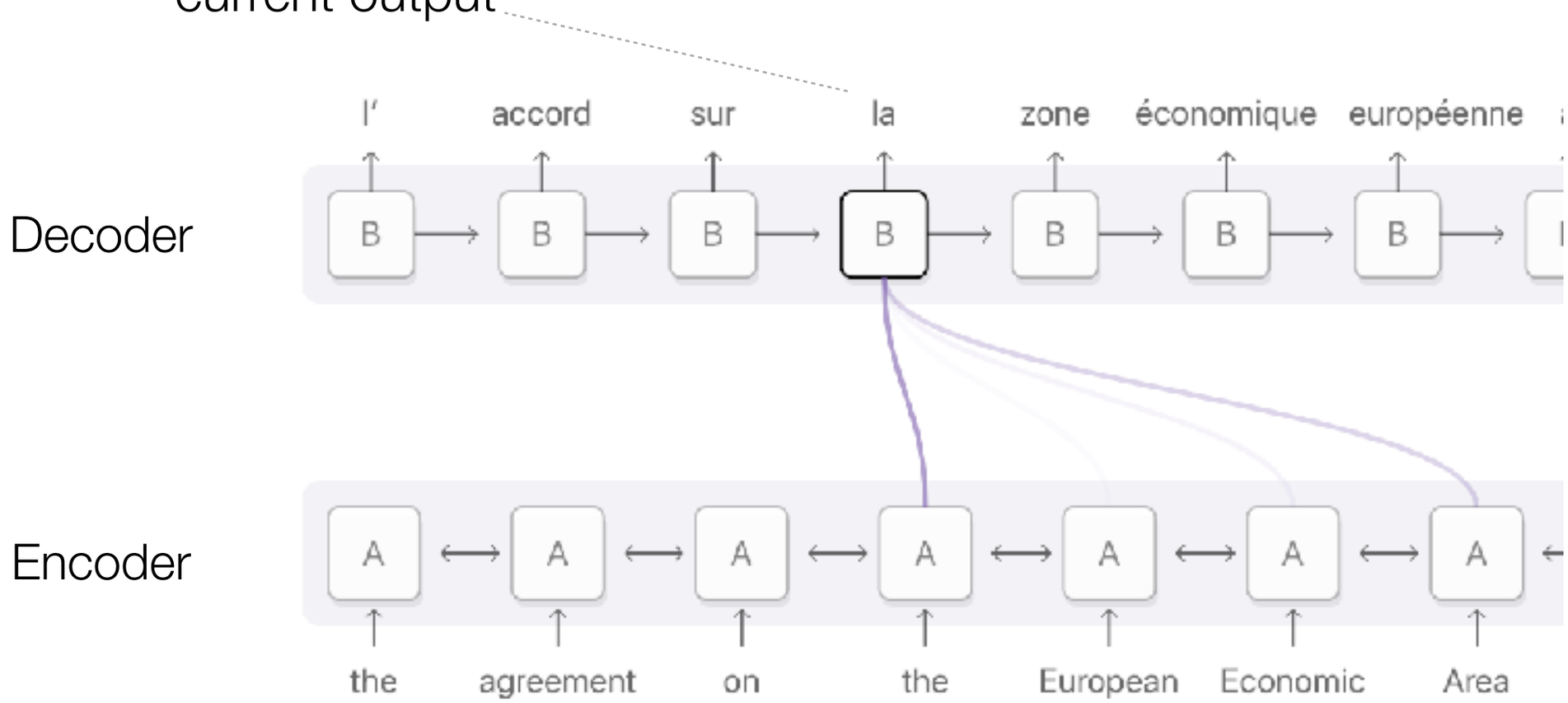


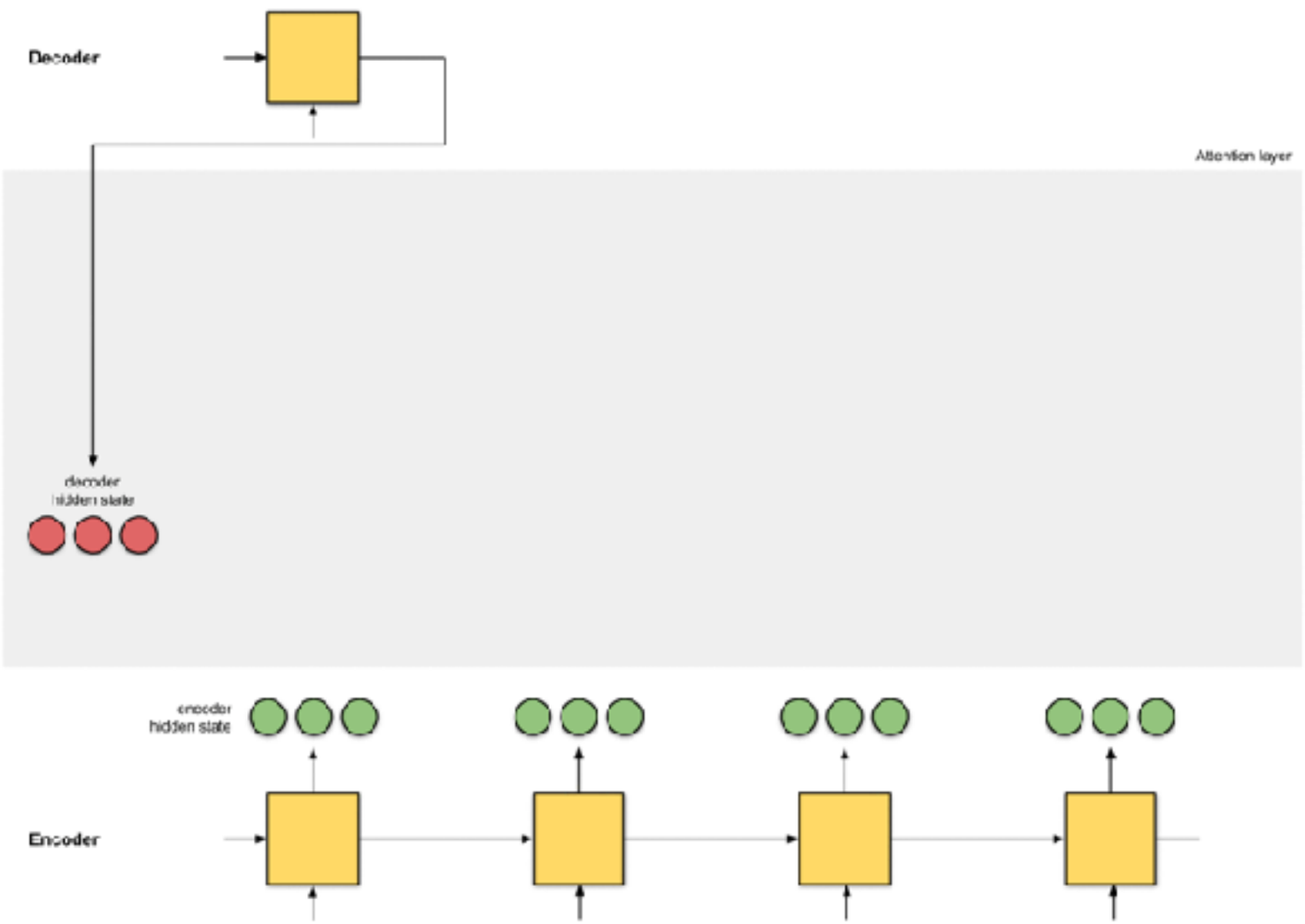
Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - intuition

- **Seq2Seq**: A translator reads the English text from start till the end. Once done, he starts translating to French word by word. It is possible that if the sentence is extremely long, he might have forgotten what he has read in the earlier parts of the text.
- **Seq2Seq with attention**: A translator reads the English text *while writing down the keywords* from the start till the end, after which he starts translating to French. While translating each English word, he makes use of the keywords he has written down.

Explanation borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

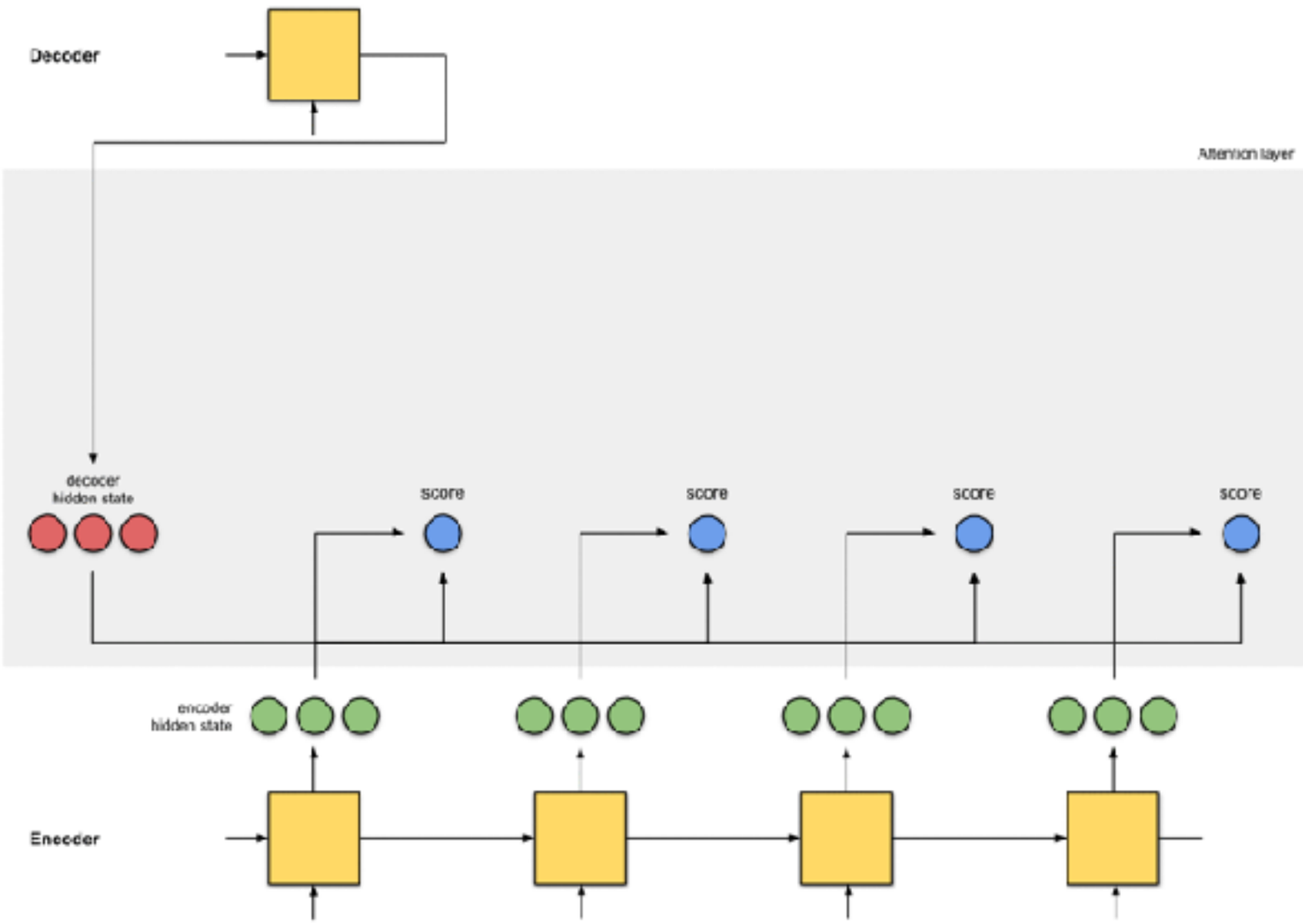
Attention - step 0: prepare hidden states



Prepare all the available encoder hidden states (green) and the first decoder hidden state (red)

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 1: obtain a score for every encoder hidden state



A **score** (scalar value) is obtained by a **score function**, aka *alignment score function* or *alignment model*.

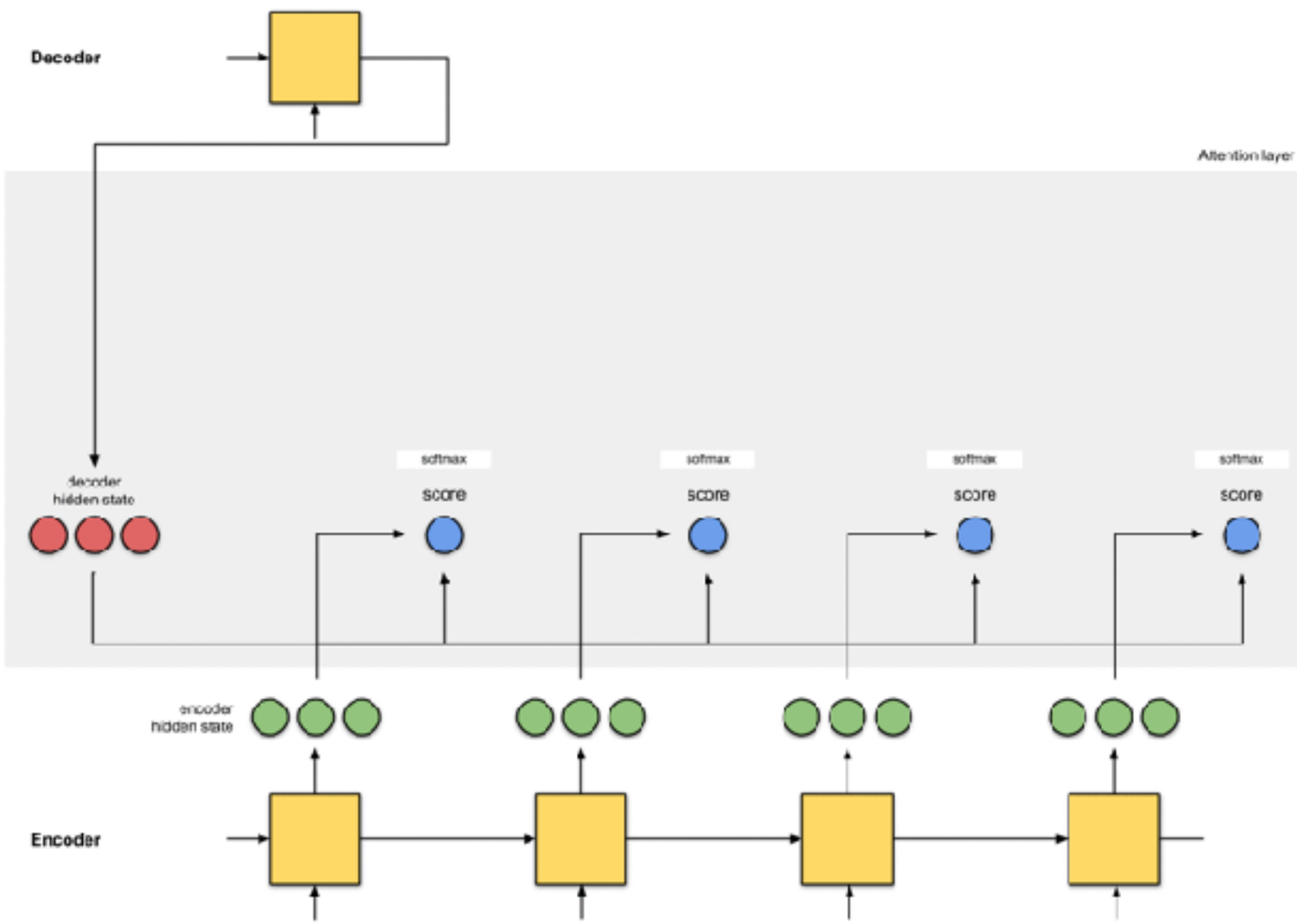
For example, the score function can be a dot product between the decoder and encoder hidden states.

```
decoder_hidden = [10, 5, 10]

encoder_hidden  score
---------------------
     [0, 1, 1]     15 (= 10×0 + 5×1 + 10×1, the dot product)
     [5, 0, 1]     60
     [1, 1, 0]     15
     [0, 5, 1]     35
```

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 2: run all the scores through a softmax layer



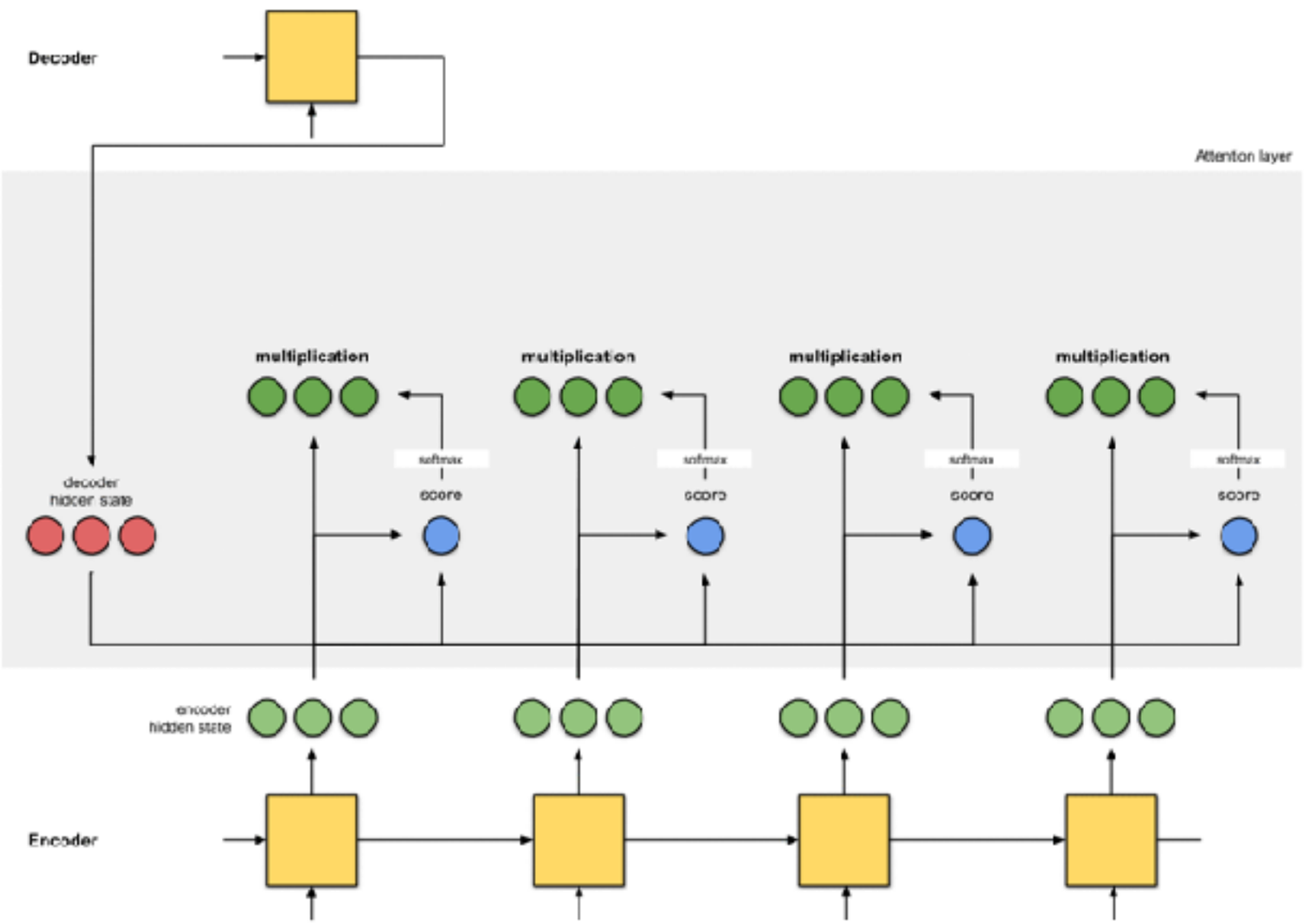
We put the scores to a softmax layer so that the softmaxed scores (scalar) add up to 1.

These softmaxed scores represent the *attention distribution.*

encoder_hidden	score	score^
[0, 1, 1]	15	0
[5, 0, 1]	60	1
[1, 1, 0]	15	0
[0, 5, 1]	35	0

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 3: Multiply each encoder hidden state by its softmaxed score



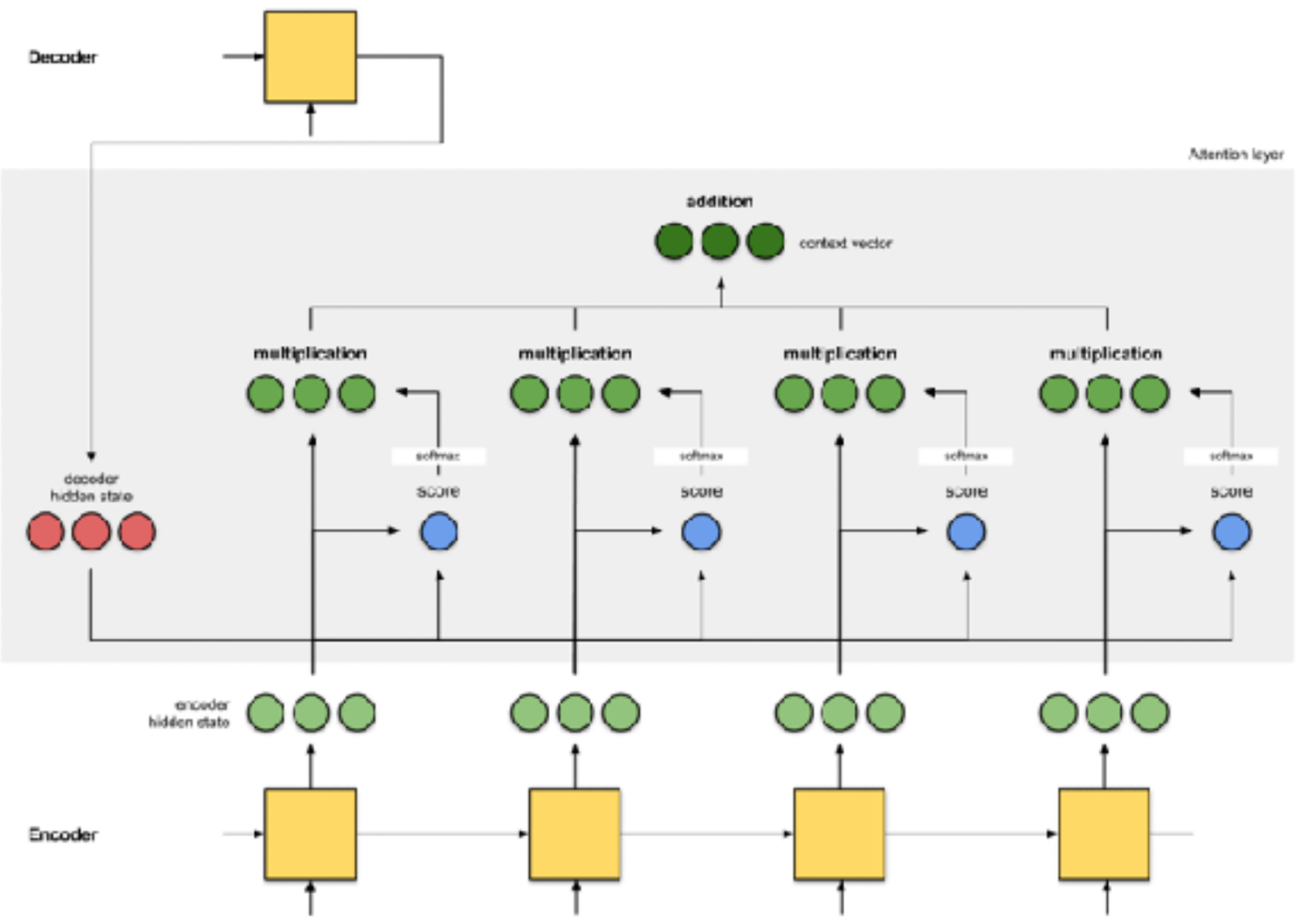
By multiplying each encoder hidden state with its softmaxed score (scalar), we obtain "weighted" versions of encoder hidden states. so-called the *alignment vectors*.

This is exactly the mechanism where alignment takes place.

encoder	score	score^	alignment
[0, 1, 1]	15	0	[0, 0, 0]
[5, 0, 1]	60	1	[5, 0, 1]
[1, 1, 0]	15	0	[0, 0, 0]
[0, 5, 1]	35	0	[0, 0, 0]

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 4: Sum up the alignment vectors.



The alignment vectors are summed up to produce the *context vector*.

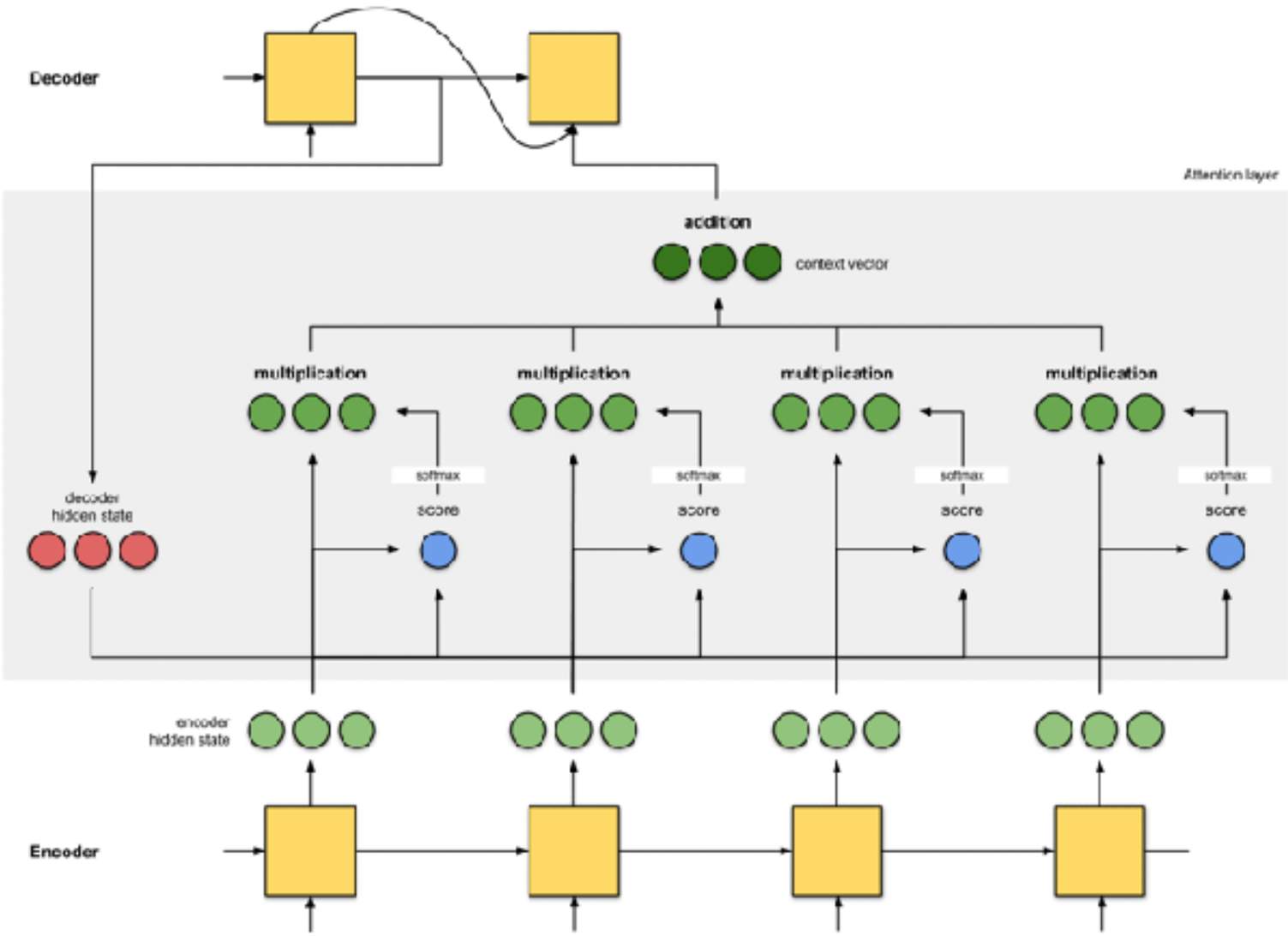
A context vector is an aggregated information of the alignment vectors from the previous step.

encoder	score	score^	alignment
[0, 1, 1]	15	0	[0, 0, 0]
[5, 0, 1]	60	1	[5, 0, 1]
[1, 1, 0]	15	0	[0, 0, 0]
[0, 5, 1]	35	0	[0, 0, 0]

context = [0+5+0+0, 0+0+0+0, 0+1+0+0] = [5, 0, 1]

Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Attention - step 5: Feed the context vector into the decoder

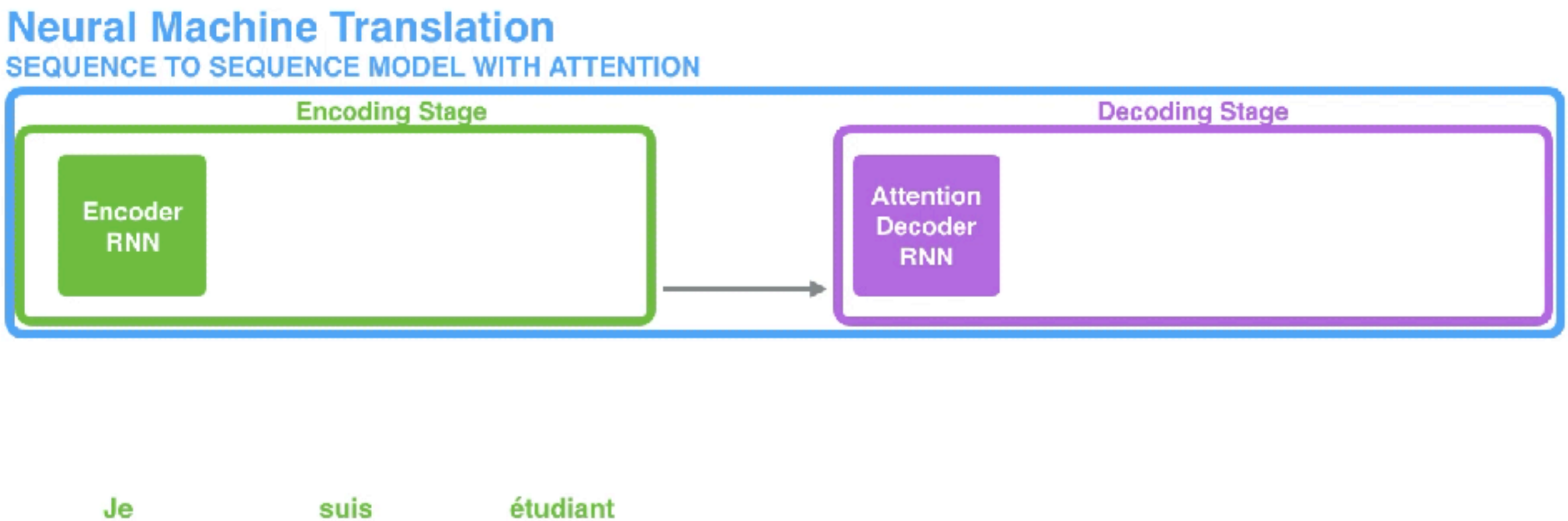


The manner this is done depends on the architecture design.

- Bahdanau et. al (2015) = concatenation of hidden state and context vector

At the next step, the decoder hidden state is used to recompute the next context vector

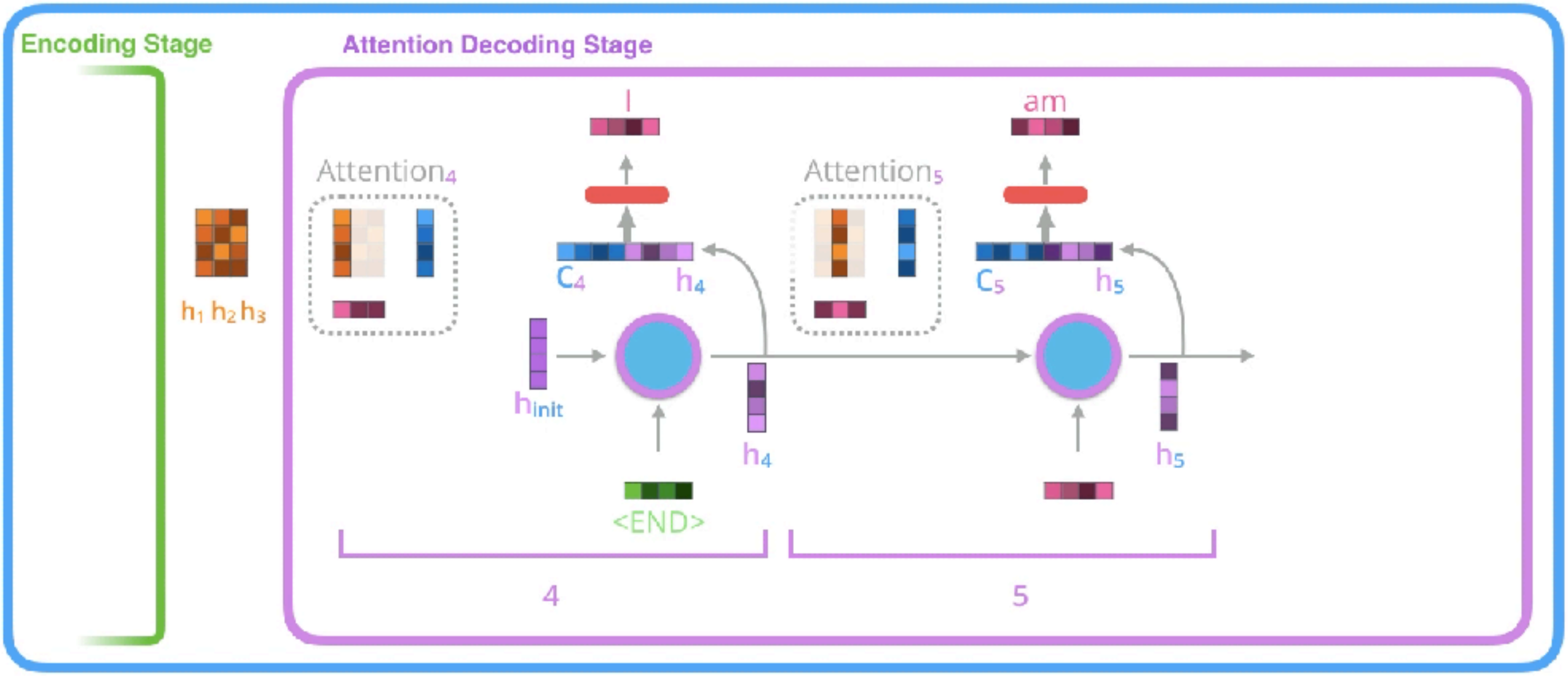
Image borrowed from https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3



Video borrowed from https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Neural Machine Translation

SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Video borrowed from https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/

Attention - further notes

- Backpropagation will do whatever it takes to ensure that the outputs will be close to the ground truth. This is done by altering the weights in the RNNs and in the score function, if any. These weights will affect the encoder hidden states and decoder hidden states, which in turn affect the attention scores.
- In step 1, different score functions can be tried: (i) additive/concat, (ii) dot product, (iii) location-based, and (iv) 'general'
- Alternate architectures have been built on top of what is explained before: https://towardsdatascience.com/attn-illustrated-attention-5ec4ad276ee3

Wrap-up

- A **Seq2Seq** model is learning to map sequences of different length in situations where the entire input sequence is required in order to start predicting the target sequence.
  - Encoder - Decoder architecture with 2 RNNs
  - h_t is passed from the last step of the encoder as input to the decoder
- In **Seq2Seq + attention,** attention is an interface between the encoder and decoder that provides the decoder with information from all (or parts of) the encoder hidden states.